

CI/CD文档

负责人：叶潮洋

概述

本文档汇总了项目中Task1和Task2的CI/CD及容器化架构方案。基于业务复杂度的不同，两个Task采用了独立的流水线设计与部署策略。

Task1：砚台AI网文生成平台

Task1采用轻量化、高效率的CI/CD流程，核心工具链基于 `uv` 和 `Node22`，并针对校园网网络环境进行了深度适配。

1.流水线架构

整体包含3个阶段，按路径变更及MergeRequest/Push事件触发：

- **Validate**：静态代码检查与类型校验。
- **Test**：单元测试。
- **Build**：前端静态文件构建与后端环境验证。

2.阶段详细配置

- **Validate阶段**
 - **frontend-lint**：基于 `node:22-alpine`，缓存 `npm` 依赖，执行 `npmrunlint`。
 - **backend-lint**：基于 `python:3.13-slim`，利用 `uv` 基于 `pyproject.toml` 缓存安装依赖并同步。按序执行 `ruffcheck`、`ruffformat--check` 以及 `mypy` 严格模式检查。
- **Test阶段**
 - **backend-test**：依赖lint阶段，执行 `pytest-q--maxfail=1`，当前34条用例已全部通过。后端覆盖率长期超过70%。
- **Build阶段**
 - **frontend-build**：执行Vite构建，将 `frontend/dist/` 目录作为产物归档，保留1周。
 - **backend-build**：验证后端运行环境就绪状态。

3.代码质量保障

- **Mypy严格模式**：在 `pyproject.toml` 开启 `strict=true`，禁用了除 `ignore_missing_imports` 外的所有豁免规则。修复了约300处类型错误。
- **Ruff规范**：统一120列行宽，开启E、F、I、N、W、UP规则集，强制进行import排序和语法现代化（如 `UP038`）。

4.国内网络环境适配

因部署宿主机处于校园网环境，进行了以下拦截绕过与加速配置：

- **PyPI**：使用 `uv` 安装时注入 `--trusted-hostmirrors.nju.edu.cn`，并配置备用回退逻辑 `||pipinstalluv`。
- **NPM**：避开丢包的南大源，统一将Registry切换为淘宝镜像 `https://registry.npmirror.com/`，解决 `@iconify/vue` 404报错。
- **Docker镜像**：放弃南大及DaoCloud镜像源（因为存在403Forbidden和SSL拦截），通过修改宿主机 `/etc/docker/daemon.json` 使用可用社区源解决基础镜像拉取失败问题。

5.Docker部署架构

采用双容器精简编排，解决与Task2的冲突：

- **前端(Nginx+Vue3)**：端口映射3000->80，Nginx配置 `/api/*` 反向代理至后端，关闭 `proxy_buffering` 以支持大模型SSE流式输出。
- **后端(FastAPI+LangGraph)**：宿主机映射端口修改为8001（避开Task2占用的8000端口）。使用SQLite持久化数据，挂载 `novel_data` 卷。同时修复了Hatchling打包时缺少README导致构建失败的问题。

Task2：全流程评测系统

Task2采用企业级重度集成的CI/CD流水线，包含9个阶段、25个并行作业，实现了从代码检查、安全审计到多环境远程部署的完整闭环。

1.流水线架构

完整链路为：`precheck->validate->test->verify->build->integration->report->package->deploy`。
各阶段采用强门禁策略，任一作业失败即阻断后续流程。

2.阶段详细配置

- **Precheck阶段**
 - 校验 `.gitlab-ci.yml` 自身配置的合法性，断言stages数量不少于6个。
- **Validate阶段**
 - **前端**：基于 `node:20-alpine`，执行构建和基础测试。
 - **后端**：基于 `python:3.11-slim` 和 `pip` 缓存。执行ruff语法/格式检查、mypy类型检查及Python字节码编译验证。
- **Test阶段**
 - **后端主测试**：执行Pytest生成Unit和Cobertura覆盖率报告，支持MR页面直观展示。
 - **API契约与插件测试**：拆分为独立作业，分别验证路由契约和核心Scoring逻辑。
 - **安全审计**：利用 `pip-audit` 扫描依赖漏洞，允许失败不阻断。
- **Verify阶段**
 - **数据库迁移**：创建临时SQLite，执行Alembic全量 `upgrade` 与 `downgrade`，验证双向迁移的健壮性。

- **ORMSchema校验**: 绕过Alembic直接建表, 断言核心表及必填字段是否完整。
- **前端审计**: 使用 `npm audit` 检查高危漏洞。
- **Build&Integration阶段**
 - 导出后端OpenAPI JSON规范文件。
 - 启动内存级测试客户端, 进行全链路API冒烟测试, 涵盖任务创建、执行、结果获取。
- **Report&Package阶段**
 - 汇总CI状态与静态资源清单。
 - 利用Docker-in-Docker并行构建前后端镜像, 并推送至私有Registry。
- **Deploy阶段**
 - **远程部署**: 通过CI变量注入密钥, 使用SSH登录目标服务器拉取镜像并触发滚动更新。支持手动触发控制。
 - **本地部署**: ShellRunner强制同步分支代码后执行 `dockercomposeup-d--build`。

3.容器化架构

采用微服务架构:

- **MySQL8.0**: 核心关系型数据库, 配置healthcheck。
- **Redis7**: 用作缓存与Celery消息代理。
- **Backend(FastAPI)**: 主业务接口 (端口8000) 。
- **Worker(Celery)**: 异步处理RAGAS评测等高耗时任务。
- **Frontend(Node/Vite)**: 前端开发服务 (端口4173) 。
- **Nginx1.27**: 总流量入口, 负责前后端路由分发。所有服务均配置了 `restart:unless-stopped` 策略。

Gitlab runner搭建与部署

由于 Task 1 和 Task 2 的环境隔离与部署需求不同, 系统为两个项目分别配置了双 Runner 架构 (Docker 执行器 + Shell 执行器), 采用了物理机部署与双执行器并行的策略。

宿主机环境: `ubuntu` 。

每个项目均配置了两种类型的 Runner 以满足不同阶段的需求:

1. Docker Executor (用于前端/后端构建与测试): 提供纯净、隔离的执行环境, 保证每次构建的一致性。
2. Shell Executor (专门用于本地自动化部署): 允许直接在宿主机 (物理机) 上执行 Bash 命令 (如 `git pull`, `docker compose up`), 实现最终产物的物理落盘与服务拉起。

搭建与配置流程

1. 基础服务安装与宿主机授权
 - 安装 GitLab Runner 核心服务
 - 将 Runner 加入 docker 用户组
 - 赋予部署目录的读写权限
2. Runner 注册与标签绑定

- 获取 Token: 在 GitLab 各个项目的 Settings -> CI/CD -> Runners 获取专属注册 Token。
3. 新建 Runner 默认运行在受保护的分支上。为支持开发分支的调试, 取消勾选 Protected 选项。
4. 修改了 Docker 源后执行 `docker builder prune -a -f` 毙掉后台幽灵缓存, 防止构建时依然请求旧的无效镜像站。

两个任务的 Runner 分配清单

Task 1 (AI 网文生成平台)

- **Docker Runner** (用于 Validate / Test / Build 阶段):
 - ID: #521
 - Token/Name: xPtTq9RNe / 260501makeForSE3Iteration1
 - Tag: 无 (或通用 tag), 作为项目的默认执行器。
- **Shell Runner** (可用于 Deploy 阶段):
 - ID: #492
 - Token/Name: dsn6NYupm / 260414makeForSE3task1
 - Tag: deploy-shell-task1 (CI YAML 中通过此 Tag 显式调用)。

Task 2 (评测系统)

- **Docker Runner** (用于前端/后端质量检测与构建):
 - ID: #477
 - Token/Name: YXf8wYLc6 / 260411makeForSE3Iteration2
 - Tag: docker ubuntu (或通用 tag)。
- **Shell Runner** (用于最终物理机部署拉起):
 - ID: #494
 - Token/Name: u-pG_ayNS / DeployRunner4Shell
 - Tag: deploy-shell (CI YAML 中通过此 Tag 显式调用)。